

Appendix A. The MicroJava Language

This section describes the MicroJava language that is used in the practical part of the compiler construction module. MicroJava is similar to Java but much simpler.

A.1 General Characteristics

- ③ A MicroJava program consists of a single program file with static fields and static methods. There are no external classes but only inner classes that can be used as data types.
- ③ The main method of a MicroJava program is always called *main()*. When a MicroJava program is called this method is executed.
- ③ There are
 - Constants of type *int* (e.g. 3) and *char* (e.g. 'x') but no string constants. -Variables: all variables of the program are static.
 - Primitive types: *int*, *char* (Ascii)
 - Reference types: one-dimensional arrays like in Java as well as classes with fields but without methods.
 - Static methods in the main class.
- ③ There is no garbage collector (allocated objects are only deallocated when the program ends).
- ③ Predeclared procedures are *ord*, *chr*, *len*.

Sample program

```
program P
  final int size = 10;

  class Table {
    int[] pos;
    int[] neg;
  }

  Table val;

  {
    void main()
    int x, i;
    { //----- Initialize val -----
      val = new Table;
      val.pos = new int[size];
      val.neg = new int[size];
      i = 0;
      while (i < size) {
        val.pos[i] = 0; val.neg[i] = 0;
        i = i + 1;
      }
      //----- Read values -----
      read(x);
      while (x != 0) {
        if (x >= 0) {
          val.pos[x] = val.pos[x] + 1;
        } else if (x < 0) {
          val.neg[-x] = val.neg[-x] + 1;
        }
      }
    }
  }
```

```

    read(x);
  }
}

```

A.2 Syntax

```

Program = "program" ident {ConstDecl | VarDecl | ClassDecl}
        "{" {MethodDecl} "}".

ConstDecl = "final" Type ident "=" (number | charConst) ";".
VarDecl = Type ident {"," ident } ";".
ClassDecl = "class" ident "{" {VarDecl} "}".
MethodDecl = (Type | "void") ident "(" [FormPars] ")" {VarDecl} Block.
FormPars = Type ident {"," Type ident}.
Type = ident "[" "[" "]" ".

Block = "{" {Statement} "}".
Statement = Designator ("=" Expr | ActPars) ";"
           | "if" "(" Condition ")" Statement ["else" Statement]
           | "while" "(" Condition ")" Statement
           | "return" [Expr] ";"
           | "read" "(" Designator ")" ";"
           | "print" "(" Expr ["," number] ")" ";"
           | Block
           | ";".
ActPars = "(" [ Expr {"," Expr} ] ")".

Condition = Expr Relop Expr.
Relop = "==" | "!=" | ">" | ">=" | "<" | "<=".
Expr = ["-"] Term {Addop Term}.
Term = Factor {Mulop Factor}.
Factor = Designator [ActPars]
        | number
        | charConst
        | "new" ident "[" [ Expr "]"
        | "(" Expr ")".
Designator = ident {"." ident | "[" Expr "]"}.
Addop = "+" | "-".
Mulop = "*" | "/" | "%".

```

Lexical structure

Character classes: letter = 'a'..'z' | 'A'..'Z'.
 digit = '0'..'9'.
 whiteSpace = ' ' | '\t' | '\r' | '\n'.

Terminal classes: ident = letter {letter | digit}.
 number = digit {digit}.
 charConst = "'" char "'". // including '\r', '\t', '\n'

Keywords: program class
 if else while read print return void final new

Operators: + - * / %
 == != > >= < <= () [] { } = ; , .

Comments: // to the end of line

A.3 Semantics

All terms in this document that have a definition are underlined to emphasize their

special meaning. The definitions of these terms are given here.

Reference type

Arrays and classes are called reference types.

Type of a constant

- ③ The type of an integer constant (e.g. 17) is `int`.
- ③ The type of a character constant (e.g. 'x') is `char`.

Same type

Two types are the same

- ③ if they are denoted by the same type name, or
- ③ if both types are arrays and their element types are the same.

Type compatibility

Two types are compatible

- ③ if they are the same, or
- ③ if one of them is a reference type and the other is the type of `null`.

Assignment compatibility

A type `src` is assignment compatible with a type `dst`

- ③ if `src` and `dst` are the same, or
- ③ if `dst` is a reference type and `src` is the type of `null`.

Predeclared names

`int` the type of all integer values

`char` the type of all character values

`null` the null value of a class or array variable, meaning "pointing to no value"

`chr` standard method; `chr(i)` converts the `int` expression `i` into a `char` value

`ord` standard method; `ord(ch)` converts the `char` value `ch` into an `int` value

`len` standard method; `len(a)` returns the number of elements of the array `a`

Scope

A scope is the textual range of a method or a class. It extends from the point after the declaring method or class name to the closing curly bracket of the method or class declaration. A scope excludes other scopes that are nested within it. We assume that there is an (artificial) outermost scope (called the *universe*), to which the main class is local and which contains all predeclared names. The declaration of a name in an inner scope hides the declarations of the same name in outer scopes.

Note

- ③ Indirectly recursive methods are not allowed, since every name must be declared before it is used. This would not be possible if indirect recursion were allowed.
- ③ A predeclared name (e.g. `int` or `char`) can be redeclared in an inner scope (but this is not recommended).

A.4 Context Conditions

General context conditions

- ③ Every name must be declared before it is used.
- ③ A name must not be declared twice in the same scope.
- ③ A program must contain a method named *main*. It must be declared as a void method and must not have parameters.

Context conditions for standard methods

chr(e) *e* must be an expression of type *int*.

ord(c) *c* must be of type *char*.

len(a) *a* must be an *array*.

Context conditions for the MicroJava productions

Program = "program" ident {ConstDecl | VarDecl | ClassDecl} "{" {MethodDecl} "}".

ConstDecl = "final" Type ident "=" (number | charConst) ";".

- ③ The type of *number* or *charConst* must be the same as the type of

Type. **VarDecl** = Type ident {" , " ident } ";".

ClassDecl = "class" ident "{" {VarDecl} "}".

MethodDecl = (Type | "void") ident "(" [FormPars] ")" {VarDecl} "{" {Statement} "}".

- ③ If a method is a function it must be left via a return statement (this is checked at run time).

FormPars = Type ident {" , " Type ident}.

Type = ident "[" "[" "]"].

- ③ *ident* must denote a type.

Statement = **Designator** "=" **Expr** ";".

- ③ *Designator* must denote a variable, an array element or an object field. ③ The type of *Expr* must be assignment compatible with the type of *Designator*.

Statement = **Designator** **ActPars** ";".

- ③ *Designator* must denote a method.

Statement = "read" "(" **Designator** ")" ";".

- ③ *Designator* must denote a variable, an array element or an object field. ③ *Designator* must be of type *int* or *char*.

Statement = "print" "(" **Expr** [" , " number] ")" ";".

- ③ *Expr* must be of type *int* or *char*.

Statement = "return" [**Expr**] .

- ③ The type of *Expr* must be assignment compatible with the function type of the current method.

③ If *Expr* is missing the current method must be declared as void.

**Statement = "if" "(" Condition ")" Statement ["else" Statement]
| "while" "(" Condition ")" Statement
| "{" {Statement} }"
| ";".**

ActPars = "(" [Expr {"," Expr}] ")".

③ The numbers of actual and formal parameters must match.

③ The type of every actual parameter must be assignment compatible with the type of every formal parameter at corresponding positions.

Condition = Expr Relop Expr.

③ The types of both expressions must be compatible.

③ Classes and arrays can only be checked for equality or inequality.

Expr = Term.

Expr = "-"Term.

③ *Term* must be of type *int*.

Expr = Expr Addop Term.

③ *Expr* and *Term* must be of type *int*.

Term = Factor.

Term = Term Mulop Factor.

③ *Term* and *Factor* must be of type *int*.

Factor = Designator | number | charConst| "(" Expr ")".

Factor = Designator ActPars.

③ *Designator* must denote a method.

Factor = "new" Type .

③ *Type* must denote a class.

Factor = "new" Type "[" Expr "]".

③ The type of *Expr* must be *int*.

Designator = Designator "." ident .

③ The type of *Designator* must be a class.

③ *ident* must be a field of *Designator*.

Designator = Designator "[" Expr "]".

③ The type of *Designator* must be an array.

③ The type of *Expr* must be *int*.

Relop = "==" | "!=" | ">" | ">=" | "<" | "<=".

Addop = "+" | "-".

Mulop = "*" | "/" | "%".

A.5 Implementation Restrictions

- ③ There must not be more than 127 local variables.
- ③ There must not be more than 32767 global variables.
- ③ A class must not have more than 32767 fields.